

Part 7 - Lists in Python

Topics Covered

- What is a List?
- Why Do We Need Lists?
- Characteristics of Lists
- List vs Array
- Creating Lists
- Accessing List Elements
- Indexing
- Negative Indexing
- List Slicing
- Modifying List Elements
- Adding Elements
 - `append()`
 - `insert()`
 - `extend()`
- Removing Elements
 - `remove()`
 - `pop()`
 - `del`
 - `clear()`
- Looping Through Lists
- List Operations
- Built-in Functions
- List Methods
- Copying Lists
- Joining Lists
- Nested Lists
- Introduction to Matrix
- List Comprehension (Introduction)

Learning Objectives

By the end of this lesson, you will be able to:

- Understand what lists are and why they are used.
- Create lists using different methods.
- Access list elements using indexing and slicing.
- Modify, add, and remove list elements.
- Iterate through lists using loops.
- Perform common list operations.
- Use Python's built-in list methods.

- Copy and join lists correctly.
- Work with nested lists.
- Create simple lists using list comprehensions.
- Write clean and efficient programs using lists.

What is a List?

A **list** is a built-in Python data structure used to store **multiple items in a single variable**.

Lists are one of the most commonly used and versatile data types in Python. They allow you to organize, store, and manipulate collections of data efficiently.

Unlike variables that hold only one value, a list can hold **many values** at the same time.

A list can contain:

- Numbers
 - Strings
 - Boolean values
 - Other lists
 - Objects
 - Mixed data types
-

Real-Life Example

Imagine you are going shopping.

Instead of carrying every item separately, you place everything into a **shopping cart**.

For example:

- Milk
- Bread
- Eggs
- Butter
- Juice

The shopping cart holds multiple items in one place.

Similarly, a **Python list** stores multiple values in a single variable.

Syntax

```
list_name = [item1, item2, item3, ...]
```

Syntax Explanation

- `list_name` → Name of the list.
 - `[ ]` → Square brackets are used to create a list.
 - Items are separated using commas.
-

Example 1: List of Numbers

```
In [ ]: numbers = [10, 20, 30, 40, 50]

print(numbers)
```

Output

```
[10, 20, 30, 40, 50]
```

Explanation

The list `numbers` contains five integer values.

Example 2: List of Strings

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

print(fruits)
```

Output

```
['Apple', 'Banana', 'Mango']
```

Explanation

Lists can store multiple string values.

Example 3: Mixed Data Types

```
In [ ]: data = ["Alice", 20, 95.5, True]

print(data)
```

Output

```
['Alice', 20, 95.5, True]
```

Explanation

A list can contain different data types together.

Example 4: Nested List

```
In [ ]: students = [
    ["Alice", 20],
    ["Bob", 21],
    ["Charlie", 22]
]

print(students)
```

Output

```
[['Alice', 20], ['Bob', 21], ['Charlie', 22]]
```

Explanation

A list can also contain other lists. This is called a **nested list**.

Where are Lists Used?

Lists are used in many real-world applications, such as:

- Storing student names.
 - Managing employee records.
 - Shopping carts in e-commerce websites.
 - Marks of students.
 - Product catalogs.
 - Sensor readings.
 - Machine Learning datasets.
-

Advantages of Lists

- Store multiple values in one variable.
 - Easy to access and modify elements.
 - Can grow or shrink dynamically.
 - Can store different data types.
 - Support many built-in methods for data manipulation.
-

Key Points

- A list is a collection of multiple items.
- Lists are created using square brackets `[]`.
- Elements are separated by commas.
- Lists can store duplicate values.
- Lists can contain different data types.
- Lists can even contain other lists.
- Lists are one of the most widely used data structures in Python.

Why Do We Need Lists?

Imagine you want to store the marks of five students.

Without a List

```
In [ ]: mark1 = 85
mark2 = 90
mark3 = 78
mark4 = 95
mark5 = 88
```

Although this works, managing many variables becomes difficult.

What if there are:

- 100 students?
- 1,000 students?
- 10,000 students?

Creating separate variables for each value is inefficient and difficult to maintain.

Using a List

```
In [ ]: marks = [85, 90, 78, 95, 88]

print(marks)
```

Output

```
[85, 90, 78, 95, 88]
```

Now all marks are stored in a single variable.

Another Example

Without a list:

```
In [ ]: fruit1 = "Apple"
fruit2 = "Banana"
fruit3 = "Mango"
fruit4 = "Orange"
```

Using a list:

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Orange"]
```

This is shorter, cleaner, and easier to manage.

Why Use Lists?

Lists help us:

- Store multiple values in one variable.
 - Reduce the number of variables.
 - Organize related data.
 - Easily access, modify, and delete elements.
 - Iterate through data using loops.
 - Perform operations like sorting, searching, copying, and joining.
-

Real-Life Example

Think about your phone's contact list.

Instead of creating separate files for each contact, all contacts are stored in one collection.

Similarly, Python lists store multiple related values together.

Advantages of Using Lists

- Less code.
 - Better organization.
 - Easier to read.
 - Easier to maintain.
 - Supports powerful built-in methods.
 - Makes looping through data simple.
-

Key Points

- Lists store multiple values in a single variable.
- They reduce code duplication.
- Lists make programs easier to read and maintain.
- They are ideal for storing collections of related data.
- Lists are one of the most frequently used data structures in Python.

Characteristics of Lists

Python lists have several important characteristics that make them one of the most flexible and widely used data structures.

The main characteristics of a list are:

1. Ordered
 2. Mutable
 3. Allows Duplicates
 4. Heterogeneous
 5. Dynamic Size
-

1. Ordered

Lists are **ordered**, which means the elements are stored in a specific sequence.

Each element has a fixed position called an **index**.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
print(fruits)
```

Output

```
['Apple', 'Banana', 'Mango']
```

Explanation

The elements remain in the same order in which they were added.

2. Mutable

Lists are **mutable**, which means their elements can be changed after the list is created.

You can:

- Modify elements
- Add elements
- Remove elements

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

fruits[1] = "Orange"

print(fruits)
```

Output

```
['Apple', 'Orange', 'Mango']
```

Explanation

The second element "Banana" is replaced with "Orange".

3. Allows Duplicates

Lists allow duplicate values.

The same element can appear multiple times.

Example

```
In [ ]: numbers = [10, 20, 20, 30, 30]

print(numbers)
```

Output

```
[10, 20, 20, 30, 30]
```

Explanation

The values 20 and 30 appear more than once, and Python stores them without any problem.

4. Heterogeneous

A list can store elements of **different data types**.

For example, a single list can contain:

- Integers
- Strings
- Floating-point numbers

- Boolean values
- Even other lists

Example

```
In [ ]: data = ["Alice", 20, 95.5, True]

print(data)
```

Output

```
['Alice', 20, 95.5, True]
```

Explanation

The list contains four different data types.

5. Dynamic Size

Lists are **dynamic**, meaning their size can increase or decrease during program execution.

You can add or remove elements whenever needed.

Example

```
In [ ]: numbers = [10, 20, 30]

numbers.append(40)

print(numbers)
```

Output

```
[10, 20, 30, 40]
```

Explanation

The new element `40` is added to the end of the list, increasing its size.

Summary Table

Characteristic	Description
Ordered	Elements maintain their insertion order.
Mutable	Elements can be changed after creation.
Allows Duplicates	Duplicate values are permitted.
Heterogeneous	Different data types can be stored together.
Dynamic Size	Elements can be added or removed at any time.

Key Points

- Lists preserve the order of elements.

- Lists are mutable and can be modified.
- Duplicate elements are allowed.
- Lists can contain multiple data types.
- Lists automatically grow or shrink as needed.

What is an Array?

An **array** is a data structure used to store **multiple values of the same data type** in a single variable.

Each element in an array is stored in a specific position called an **index**, which starts from `0`.

Note: Python does not have a built-in array data type like C or Java. Instead, Python mainly uses **lists**. Arrays can be created using the `array` module or libraries such as NumPy.

Example (Python Array)

```
In [ ]: from array import array

numbers = array('i', [10, 20, 30, 40])

print(numbers)
```

Output

```
array('i', [10, 20, 30, 40])
```

Key Points

- Stores multiple values in one variable.
- Stores elements of the **same data type**.
- Elements are accessed using an index.
- Arrays are mainly used for numerical data.
- In Python, **lists are more commonly used than arrays**.

List vs Array

Both **lists** and **arrays** are used to store multiple values.

However, they are **not the same**.

A **list** is a built-in Python data structure that can store different types of data.

An **array** stores elements of the **same data type** and is available through modules such as the `array` module or libraries like NumPy.

Note: In Python, lists are used much more frequently than arrays because they are flexible and easy to use.

Python List

A list can store elements of different data types.

Example

```
In [ ]: data = [10, "Python", 3.14, True]

print(data)
```

Output

```
[10, 'Python', 3.14, True]
```

Explanation

The list contains an integer, a string, a floating-point number, and a boolean value.

Python Array

Python arrays are created using the `array` module.

All elements in an array must have the **same data type**.

Example

```
In [ ]: from array import array

numbers = array('i', [10, 20, 30, 40])

print(numbers)
```

Output

```
array('i', [10, 20, 30, 40])
```

Explanation

- `'i'` indicates that the array stores integers.
 - All elements must be integers.
-

Difference Between List and Array

List	Array
Built into Python.	Requires the <code>array</code> module or NumPy.
Can store different data types.	Stores only one data type.
More flexible.	More memory efficient.
Most commonly used in Python.	Commonly used for numerical data.
Supports many built-in methods.	Mainly used for mathematical computations.

When Should You Use a List?

Use a list when:

- You want to store different types of data.
- The size of the collection may change.
- You need a general-purpose collection.

Example

```
In [ ]: student = ["Alice", 20, 95.5, True]
```

When Should You Use an Array?

Use an array when:

- All elements are of the same data type.
- You are working with numerical data.
- Memory efficiency is important.

Example

```
In [ ]: from array import array  
  
marks = array('i', [85, 90, 78, 95])
```

Which One Should Beginners Learn First?

For beginners, **lists are much more important** because:

- They are built into Python.
- They are easier to learn.
- Most Python programs use lists.
- Lists are widely used in web development, automation, data analysis, and machine learning.

Arrays are typically introduced later when learning modules such as **NumPy**.

Key Points

- Both lists and arrays store multiple values.
- Lists can store different data types.
- Arrays store only one data type.
- Lists are built into Python.
- Arrays require the `array` module or libraries like NumPy.
- Beginners should focus on learning lists before arrays.

Creating Lists

There are several ways to create a list in Python.

The most common method is using **square brackets** `[]`, but Python also provides the built-in `list()` function.

1. Creating an Empty List

An empty list contains no elements.

Syntax

```
list_name = []
```

Example

```
In [ ]: numbers = []  
  
print(numbers)
```

Output

```
[]
```

Explanation

The list `numbers` is empty because it does not contain any elements.

2. Creating a List with Multiple Elements

Multiple elements are separated by commas inside square brackets.

Example

```
In [ ]: numbers = [10, 20, 30, 40, 50]  
  
print(numbers)
```

Output

```
[10, 20, 30, 40, 50]
```

Explanation

The list contains five integer values.

3. Creating a List of Strings

Lists can also store string values.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits)
```

Output

```
['Apple', 'Banana', 'Mango']
```

Explanation

Each element in the list is a string.

4. Creating a List with Mixed Data Types

A list can contain different data types.

Example

```
In [ ]: data = ["Alice", 20, 95.5, True]

print(data)
```

Output

```
['Alice', 20, 95.5, True]
```

Explanation

The list contains:

- String
 - Integer
 - Float
 - Boolean
-

5. Creating a Nested List

A list can contain other lists.

Example

```
In [ ]: students = [
    ["Alice", 20],
    ["Bob", 21],
    ["Charlie", 22]
]

print(students)
```

Output

```
[['Alice', 20], ['Bob', 21], ['Charlie', 22]]
```

Explanation

Each element of the main list is another list.

6. Creating a List Using the `list()` Function

Python provides the built-in `list()` function to create a list.

Example 1

```
In [ ]: numbers = list((10, 20, 30, 40))

print(numbers)
```

Output

```
[10, 20, 30, 40]
```

Explanation

The tuple is converted into a list.

Example 2

```
In [ ]: letters = list("Python")

print(letters)
```

Output

```
['P', 'y', 't', 'h', 'o', 'n']
```

Explanation

Each character in the string becomes a separate list element.

Example 3

```
numbers = list(range(1, 6))

print(numbers)
```

Output

```
[1, 2, 3, 4, 5]
```

Explanation

The `range()` function generates numbers from 1 to 5, and `list()` converts them into a list.

Summary Table

Method	Example
Empty List	<code>[]</code>
Multiple Elements	<code>[10, 20, 30]</code>
String List	<code>["Apple", "Banana"]</code>
Mixed Data Types	<code>["Alice", 20, True]</code>
Nested List	<code>[[1, 2], [3, 4]]</code>
<code>list()</code> Function	<code>list(range(5))</code>

Key Points

- Lists are usually created using square brackets `[]`.
- Elements are separated by commas.
- A list can be empty or contain one or more elements.
- Lists can store the same or different data types.
- Lists can contain other lists (nested lists).
- The `list()` function can create a list from other iterable objects such as strings, tuples, and ranges.

Accessing List Elements

Each element in a list has a unique position called an **index**.

Python uses **indexing** to access individual elements in a list.

Note: List indexing starts from **0**, not 1.

Syntax

```
list_name[index]
```

Syntax Explanation

- `list_name` → Name of the list.
 - `index` → Position of the element.
 - The first element has an index of `0`.
-

Index Position

Consider the following list:

```
fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]
```

Index	0	1	2	3	4
Value	Apple	Banana	Mango	Orange	Grapes

Example 1: Accessing the First Element

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]
print(fruits[0])
```

Output

Apple

Explanation

The first element has an index of `0`.

Example 2: Accessing the Second Element

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[1])
```

Output

Banana

Explanation

The second element has an index of `1`.

Example 3: Accessing the Last Element

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[2])
```

Output

Mango

Explanation

The last element is at index `2`.

Example 4: Accessing Multiple Elements

```
In [ ]: numbers = [10, 20, 30, 40, 50]  
  
print(numbers[0])  
print(numbers[2])  
print(numbers[4])
```

Output

10

30

50

Explanation

Different elements can be accessed using their respective indexes.

Example 5: Accessing Elements from a Mixed List

```
In [ ]: data = ["Alice", 20, 95.5, True]  
  
print(data[0])  
print(data[2])
```

Output

Alice

95.5

Explanation

Lists can store different data types, and each element can be accessed using its index.

What Happens If the Index Does Not Exist?

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[5])
```

Result

IndexError: list index out of range

Explanation

The list has only three elements (indexes `0`, `1`, and `2`). Trying to access index `5` causes an `IndexError`.

Key Points

- List indexing starts from `0`.
- Use square brackets `[]` to access elements.
- Each element has a unique index.
- Accessing an invalid index raises an `IndexError`.
- Indexing provides direct access to any element in the list.

Negative Indexing

In Python, you can also access list elements using **negative indexes**.

Negative indexing starts from the **end of the list**.

- `-1` → Last element
- `-2` → Second last element
- `-3` → Third last element
- and so on.

Note: Negative indexing is useful when you want to access elements from the end of a list without knowing its length.

Syntax

```
list_name[-index]
```

Syntax Explanation

- `list_name` → Name of the list.
 - `-index` → Position counted from the end of the list.
-

Index Position

Consider the following list:

```
fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]
```

Positive Index	0	1	2	3	4
Value	Apple	Banana	Mango	Orange	Grapes
Negative Index	-5	-4	-3	-2	-1

Example 1: Accessing the Last Element

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[-1])
```

Output

Mango

Explanation

`-1` always refers to the last element of the list.

Example 2: Accessing the Second Last Element

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[-2])
```

Output

Banana

Explanation

`-2` refers to the second last element.

Example 3: Accessing Multiple Elements

```
In [ ]: numbers = [10, 20, 30, 40, 50]  
  
print(numbers[-1])  
print(numbers[-3])  
print(numbers[-5])
```

Output

50
30
10

Explanation

Python counts the indexes from the end of the list.

Example 4: Accessing Elements from a Mixed List

```
In [ ]: data = ["Alice", 20, 95.5, True]

print(data[-1])
print(data[-2])
```

Output

```
True
95.5
```

Explanation

Negative indexing works with lists containing any data type.

What Happens If the Negative Index Does Not Exist?

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

print(fruits[-4])
```

Result

```
IndexError: list index out of range
```

Explanation

The list contains only three elements, so index `-4` does not exist.

When Should You Use Negative Indexing?

Use negative indexing when you want to:

- Access the last element.
 - Access the last few elements.
 - Avoid calculating the length of the list.
 - Write cleaner and more readable code.
-

Key Points

- Negative indexing starts from the end of the list.
- `-1` refers to the last element.
- `-2` refers to the second last element.
- Negative indexing works with all Python sequences, including lists, strings, and tuples.
- Accessing an invalid negative index raises an `IndexError`.

List Slicing

List slicing is used to access a portion (or subset) of a list.

Instead of accessing a single element, slicing allows you to retrieve **multiple elements** at once.

Note: Slicing creates a **new list** without modifying the original list.

Syntax

```
list_name[start : stop : step]
```

Syntax Explanation

- **start** → Starting index (inclusive).
- **stop** → Ending index (exclusive).
- **step** → Number of positions to move (optional).

If any value is omitted, Python uses a default value.

Understanding the Indexes

Consider the following list:

```
fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]
```

Positive Index	0	1	2	3	4
Value	Apple	Banana	Mango	Orange	Grapes
Negative Index	-5	-4	-3	-2	-1

1. Basic Slicing

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]
        print(fruits[1:4])
```

Output

```
['Banana', 'Mango', 'Orange']
```

Explanation

- Starts from index **1**.
- Stops before index **4**.
- Index **4** is **not included**.

2. Omitting the Start Index

If the start index is omitted, slicing begins from the first element.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]
```

```
print(fruits[:3])
```

Output

```
['Apple', 'Banana', 'Mango']
```

3. Omitting the Stop Index

If the stop index is omitted, slicing continues until the end of the list.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]  
  
print(fruits[2:])
```

Output

```
['Mango', 'Orange', 'Grapes']
```

4. Omitting Both Start and Stop

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(fruits[:])
```

Output

```
['Apple', 'Banana', 'Mango']
```

Explanation

This returns a copy of the entire list.

5. Using Step

The **step** value determines how many positions Python moves after selecting an element.

Example

```
In [ ]: numbers = [10, 20, 30, 40, 50, 60]  
  
print(numbers[::2])
```

Output

```
[10, 30, 50]
```

Explanation

Python selects every second element.

Example

```
In [ ]: numbers = [10, 20, 30, 40, 50, 60]

print(numbers[1::2])
```

Output

[20, 40, 60]

6. Negative Slicing

Negative indexes can also be used while slicing.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]

print(fruits[-4:-1])
```

Output

['Banana', 'Mango', 'Orange']

Explanation

- Starts from -4 .
 - Stops before -1 .
-

7. Reversing a List

A negative step reverses the direction of slicing.

Example

```
In [ ]: numbers = [10, 20, 30, 40, 50]

print(numbers[::-1])
```

Output

[50, 40, 30, 20, 10]

Explanation

[::-1] creates a reversed copy of the list.

8. Copying a List Using Slicing

Example

```
In [ ]: numbers = [10, 20, 30]

copy_numbers = numbers[:]
```

```
print(copy_numbers)
```

Output

```
[10, 20, 30]
```

Explanation

Slicing with `[ : ]` creates a shallow copy of the list.

Summary Table

Expression	Description
<code>list[:]</code>	Copy the entire list
<code>list[1:4]</code>	Elements from index 1 to 3
<code>list[:3]</code>	First three elements
<code>list[2:]</code>	From index 2 to the end
<code>list[::2]</code>	Every second element
<code>list[::-1]</code>	Reverse the list
<code>list[-4:-1]</code>	Slice using negative indexes

Common Mistakes

Incorrect

```
numbers = [10, 20, 30, 40]
```

```
print(numbers[1:5])
```

Output

```
[20, 30, 40]
```

Explanation

Python does **not** raise an error when the stop index is greater than the list length. It simply returns all available elements up to the end of the list.

Key Points

- Slicing returns a new list.
- The **start index is included**.
- The **stop index is excluded**.
- The **step** value controls the interval between selected elements.
- Negative indexes can be used in slicing.
- `[::-1]` is the easiest way to reverse a list.
- `[ : ]` creates a copy of the list.

Modifying List Elements

Lists in Python are **mutable**, which means you can change their elements after the list has been created.

You can:

- Modify a single element.
- Modify multiple elements.
- Replace elements using slicing.

Note: Unlike strings, lists are mutable, so their elements can be changed directly.

1. Modifying a Single Element

Use the index of the element you want to change.

Syntax

```
list_name[index] = new_value
```

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

fruits[1] = "Orange"

print(fruits)
```

Output

```
['Apple', 'Orange', 'Mango']
```

Explanation

- The element at index `1` is `"Banana"`.
 - It is replaced with `"Orange"`.
-

2. Modifying the First Element

```
In [ ]: numbers = [10, 20, 30, 40]

numbers[0] = 100

print(numbers)
```

Output

```
[100, 20, 30, 40]
```

3. Modifying the Last Element

You can also use negative indexing.

```
In [ ]: numbers = [10, 20, 30, 40]

numbers[-1] = 500

print(numbers)
```

Output

```
[10, 20, 30, 500]
```

4. Modifying Multiple Elements

List slicing allows you to replace multiple elements at once.

Syntax

```
list_name[start:stop] = [new_values]
```

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Orange"]

fruits[1:3] = ["Kiwi", "Grapes"]

print(fruits)
```

Output

```
['Apple', 'Kiwi', 'Grapes', 'Orange']
```

Explanation

The elements from index `1` to `2` are replaced with `"Kiwi"` and `"Grapes"`.

5. Replacing with More Elements

The number of new elements does not have to match the number of replaced elements.

```
In [ ]: numbers = [10, 20, 30]

numbers[1:2] = [100, 200, 300]

print(numbers)
```

Output

```
[10, 100, 200, 300, 30]
```

Explanation

One element is replaced with three elements, increasing the size of the list.

6. Replacing with Fewer Elements

```
In [ ]: numbers = [10, 20, 30, 40]

numbers[1:3] = [100]

print(numbers)
```

Output

```
[10, 100, 40]
```

Explanation

Two elements are replaced with one element, reducing the size of the list.

7. Modifying All Elements

```
In [ ]: numbers = [10, 20, 30]

numbers[:] = [1, 2, 3]

print(numbers)
```

Output

```
[1, 2, 3]
```

Explanation

The entire list is replaced with new values.

What Happens If the Index Does Not Exist?

```
In [ ]: numbers = [10, 20, 30]

numbers[5] = 100
```

Result

```
IndexError: list assignment index out of range
```

Explanation

The list contains only three elements. Since index 5 does not exist, Python raises an `IndexError`.

Summary Table

Operation	Example
Change one element	<code>list[1] = value</code>
Change last element	<code>list[-1] = value</code>
Change multiple elements	<code>list[1:3] = [...]</code>
Replace entire list	<code>list[:] = [...]</code>

Key Points

- Lists are mutable, so their elements can be modified.
- Use indexing to modify a single element.
- Use slicing to modify multiple elements.
- The replacement list can contain more or fewer elements than the original slice.
- Assigning to an invalid index raises an `IndexError`.

Adding Elements

Python lists are **dynamic**, which means you can add new elements to a list at any time.

There are several ways to add elements to a list depending on where you want to insert them.

Python provides the following methods for adding elements:

- `append()` – Adds a single element to the end of the list.
- `insert()` – Adds an element at a specific position.
- `extend()` – Adds multiple elements from another iterable.

Note: These methods modify the original list. They do not create a new list.

Methods for Adding Elements

1. `append()`

Adds a **single element** to the end of the list.

Example

```
In [ ]: fruits = ["Apple", "Banana"]

fruits.append("Mango")

print(fruits)
```

Output

```
['Apple', 'Banana', 'Mango']
```

2. `insert()`

Adds an element at a **specific index**.

Example

```
In [ ]: fruits = ["Apple", "Banana"]

fruits.insert(1, "Orange")

print(fruits)
```

Output

```
['Apple', 'Orange', 'Banana']
```

3. `extend()`

Adds **multiple elements** from another iterable such as a list, tuple, or set.

Example

```
In [ ]: fruits = ["Apple", "Banana"]

fruits.extend(["Mango", "Orange"])

print(fruits)
```

Output

```
['Apple', 'Banana', 'Mango', 'Orange']
```

Summary Table

Method	Description
<code>append()</code>	Adds one element to the end of the list.
<code>insert()</code>	Adds one element at a specified index.
<code>extend()</code>	Adds multiple elements from another iterable.

Comparision table

Feature	<code>append()</code>	<code>insert()</code>	<code>extend()</code>
Adds one element	✓	✓	✗
Adds multiple elements	✗	✗	✓
Adds at end	✓	Optional	Usually yes
Adds at specific index	✗	✓	✗

Key Points

- Lists can grow dynamically.
- Use `append()` to add one element at the end.
- Use `insert()` to add an element at a specific position.
- Use `extend()` to add multiple elements at once.
- All three methods modify the original list.

Removing Elements

Python provides several ways to remove elements from a list.

The method you choose depends on whether you want to remove an element by **value**, **index**, or **remove all elements**.

Python provides the following methods for removing elements:

- `remove()` – Removes the first occurrence of a specified value.
- `pop()` – Removes and returns an element at a specified index (or the last element by default).
- `del` – Deletes an element, a slice of elements, or the entire list.
- `clear()` – Removes all elements from the list, making it empty.

Note: These methods modify the original list.

Methods for Removing Elements

1. `remove()`

Removes the **first occurrence** of a specified value.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

fruits.remove("Banana")

print(fruits)
```

Output

```
['Apple', 'Mango']
```

2. `pop()`

Removes and returns an element at a specified index.

If no index is provided, it removes the **last element**.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

fruits.pop()

print(fruits)
```

Output

```
['Apple', 'Banana']
```

3. `del`

Deletes an element, multiple elements, or the entire list.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

del fruits[1]

print(fruits)
```

Output

['Apple', 'Mango']

4. clear()

Removes all elements from the list.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

fruits.clear()

print(fruits)
```

Output

[]

Summary Table

Method	Description
remove()	Removes the first occurrence of a specified value.
pop()	Removes and returns an element by index (last by default).
del	Deletes an element, a slice, or the entire list.
clear()	Removes all elements from the list.

Comparision table

Feature	remove()	pop()	del	clear()
Removes by value	✓	✗	✗	✗
Removes by index	✗	✓	✓	✗
Returns removed element	✗	✓	✗	✗
Deletes entire list	✗	✗	✓	✗
Empties the list	✗	✗	✗	✓

Key Points

- Use remove() to remove an element by its value.

- Use `pop()` to remove an element by its index.
- Use `del` to delete specific elements, slices, or the entire list.
- Use `clear()` to empty a list while keeping the list variable.
- All four methods modify the original list.

Looping Through Lists

Looping through a list means accessing each element of the list one by one.

Python provides several ways to iterate over a list.

The most common methods are:

- `for` loop
- `for` loop with `range()`
- `while` loop
- `enumerate()`

Note: The `for` loop is the most commonly used and recommended way to iterate through a list.

1. Using a `for` Loop

The `for` loop directly accesses each element of the list.

Syntax

```
for item in list_name:  
    # Code
```

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
for fruit in fruits:  
    print(fruit)
```

Output

```
Apple  
Banana  
Mango
```

Explanation

The loop visits each element one by one and prints it.

2. Using `range()`

The `range()` function iterates through the indexes of the list.

Syntax

```
for index in range(len(list_name)):
    # Code
```

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

for i in range(len(fruits)):
    print(fruits[i])
```

Output

Apple
Banana
Mango

Explanation

- `len(fruits)` returns the number of elements.
 - `range()` generates the indexes.
 - `fruits[i]` accesses each element using its index.
-

3. Using a `while` Loop

A `while` loop can also be used to iterate through a list.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

i = 0

while i < len(fruits):
    print(fruits[i])
    i += 1
```


Output

Apple
Banana
Mango

Explanation

The loop continues until all elements have been accessed.

4. Using enumerate()

The `enumerate()` function returns both the **index** and the **value** of each element.

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

for index, value in enumerate(fruits):
    print(index, value)
```

Output

0 Apple
1 Banana
2 Mango

Explanation

- `index` stores the position of the element.
- `value` stores the actual element.

`enumerate()` is useful when you need both the index and the value.

Summary Table

Method	When to Use
<code>for</code> loop	When only the elements are needed.
<code>range()</code>	When the indexes are needed.
<code>while</code> loop	When iteration depends on a condition.
<code>enumerate()</code>	When both the index and the element are needed.

Key Points

- The `for` loop is the simplest way to iterate through a list.
- Use `range(len(list))` when you need indexes.
- Use a `while` loop when iteration depends on a condition.
- Use `enumerate()` when you need both the index and the value.
- All these methods iterate through every element in the list.

List Operations

Python provides several operators that can be used to perform operations on lists.

The most common list operations are:

- Concatenation (`+`)
- Repetition (`*`)
- Membership (`in` , `not in`)
- Comparison (`==` , `!=` , `<` , `>`)

These operations make it easy to combine, repeat, search, and compare lists.

1. Concatenation (`+`)

The `+` operator joins two or more lists into a single list.

Syntax

```
new_list = list1 + list2
```

Example

```
In [ ]: list1 = [1, 2, 3]
list2 = [4, 5, 6]

result = list1 + list2

print(result)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation

The `+` operator combines the elements of both lists into a new list.

2. Repetition (`*`)

The `*` operator repeats the elements of a list a specified number of times.

Syntax

```
new_list = list_name * number
```

Example

```
In [ ]: numbers = [1, 2, 3]

print(numbers * 3)
```

Output

```
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Explanation

The list is repeated three times.

3. Membership Operators

Membership operators check whether an element exists in a list.

Python provides two membership operators:

- `in`
 - `not in`
-

Using `in`

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

print("Banana" in fruits)
```

Output

```
True
```

Explanation

"Banana" exists in the list, so the result is `True`.

Using `not in`

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]

print("Orange" not in fruits)
```

Output

```
True
```

Explanation

"Orange" is not present in the list, so the result is `True`.

4. Comparison Operators

Lists can be compared using comparison operators.

Python compares lists element by element.

Equality (==)

Example

```
In [ ]: list1 = [1, 2, 3]
list2 = [1, 2, 3]

print(list1 == list2)
```

True

Output

True

Explanation

Both lists contain the same elements in the same order.

Not Equal (!=)

Example

```
In [ ]: list1 = [1, 2, 3]
list2 = [1, 2, 4]

print(list1 != list2)
```

Output

True

Explanation

The lists are different, so the result is True .

Greater Than (>)

Example

```
In [ ]: list1 = [2, 3]
list2 = [1, 5]

print(list1 > list2)
```

Output

True

Explanation

Python compares the lists element by element.

- First elements: `2 > 1`
- Since the first comparison decides the result, Python returns `True` .

Less Than (<)

Example

```
In [ ]: list1 = [1, 2]
list2 = [2, 1]

print(list1 < list2)
```

Output

True

Explanation

The first element of `list1` is smaller than the first element of `list2` .

Summary Table

Operation	Operator	Description
Concatenation	<code>+</code>	Joins two or more lists.
Repetition	<code>*</code>	Repeats the list.
Membership	<code>in</code>	Checks whether an element exists.
Membership	<code>not in</code>	Checks whether an element does not exist.
Comparison	<code>==</code>	Checks if two lists are equal.
Comparison	<code>!=</code>	Checks if two lists are not equal.
Comparison	<code><</code> , <code>></code>	Compares lists element by element.

Key Points

- Use `+` to join lists.
- Use `*` to repeat a list.
- Use `in` and `not in` to check membership.
- Python compares lists element by element.
- List operations create new results without modifying the original lists (except when assigned back to a variable).

Built-in Functions

Python provides several **built-in functions** that work with lists.

Unlike **list methods**, these functions can also work with other iterable objects such as tuples, strings, and sets.

Some commonly used built-in functions for lists are:

- `len()`
 - `min()`
 - `max()`
 - `sum()`
 - `sorted()`
 - `any()`
 - `all()`
-

1. `len()`

The `len()` function returns the total number of elements in a list.

Syntax

```
len(list_name)
```

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
print(len(fruits))
```

Output

3

Explanation

The list contains three elements, so `len()` returns `3`.

2. `min()`

The `min()` function returns the smallest element in a list.

Syntax

```
min(list_name)
```

Example

```
In [ ]: numbers = [40, 10, 60, 20]  
  
print(min(numbers))
```

Output

10

Explanation

10 is the smallest value in the list.

3. max()

The `max()` function returns the largest element in a list.

Syntax

```
max(list_name)
```

Example

```
In [ ]: numbers = [40, 10, 60, 20]

print(max(numbers))
```

Output

60

Explanation

60 is the largest value in the list.

4. sum()

The `sum()` function returns the sum of all numeric elements in a list.

Syntax

```
sum(list_name)
```

Example

```
In [ ]: numbers = [10, 20, 30, 40]

print(sum(numbers))
```

Output

100

Explanation

The function adds all the numbers in the list.

5. sorted()

The `sorted()` function returns a **new sorted list**.

It **does not modify** the original list.

Syntax

```
sorted(list_name)
```

Example

```
In [ ]: numbers = [40, 10, 60, 20]

print(sorted(numbers))
```

Output

```
[10, 20, 40, 60]
```

Explanation

The function returns a new list in ascending order.

Descending Order

```
In [ ]: numbers = [40, 10, 60, 20]

print(sorted(numbers, reverse=True))
```

Output

```
[60, 40, 20, 10]
```

6. any()

The `any()` function returns **True** if **at least one element** in the list is **True**.

Otherwise, it returns **False**.

Syntax

```
any(list_name)
```

Example

```
In [ ]: values = [False, False, True]

print(any(values))
```

Output

```
True
```


Explanation

Since one element is `True` , `any()` returns `True` .

7. `all()`

The `all()` function returns `True` only if **all elements** in the list are `True` .

If even one element is `False` , it returns `False` .

Syntax

```
all(list_name)
```

Example

```
In [ ]: values = [True, True, False]
        print(all(values))
```

Output

False

Explanation

Since one element is `False` , `all()` returns `False` .

Summary Table

Function	Description
<code>len()</code>	Returns the number of elements.
<code>min()</code>	Returns the smallest element.
<code>max()</code>	Returns the largest element.
<code>sum()</code>	Returns the sum of numeric elements.
<code>sorted()</code>	Returns a new sorted list.
<code>any()</code>	Returns <code>True</code> if at least one element is <code>True</code> .
<code>all()</code>	Returns <code>True</code> if all elements are <code>True</code> .

Built-in Functions vs List Methods

Built-in Functions	List Methods
Called using the function name.	Called using the list object.
Work with many iterable objects.	Work only with lists.

Example: `len(list1)`Example: `list1.append(10)`

Key Points

- Built-in functions work with lists and other iterable objects.
- `len()` returns the list length.
- `min()` and `max()` return the smallest and largest elements.
- `sum()` adds all numeric elements.
- `sorted()` returns a new sorted list without modifying the original list.
- `any()` checks if at least one element is `True`.
- `all()` checks if every element is `True`.

List Methods

Python provides several built-in methods that make it easy to work with lists.

List methods can be used to:

- Search elements
- Count elements
- Sort elements
- Reverse a list
- Copy a list
- Add or remove elements

Note: List methods are called using the **dot (.) operator**.

Common List Methods

Method	Description
<code>append()</code>	Adds an element to the end of the list.
<code>insert()</code>	Inserts an element at a specified index.
<code>extend()</code>	Adds all elements from another iterable.
<code>remove()</code>	Removes the first occurrence of a specified value.
<code>pop()</code>	Removes and returns an element by index.
<code>clear()</code>	Removes all elements from the list.
<code>index()</code>	Returns the index of the first occurrence of a value.
<code>count()</code>	Counts the number of occurrences of a value.
<code>sort()</code>	Sorts the list in ascending order.
<code>reverse()</code>	Reverses the order of the list.
<code>copy()</code>	Returns a shallow copy of the list.

1. index()

The `index()` method returns the index of the **first occurrence** of the specified value.

Syntax

```
list_name.index(value)
```

Example

```
In [ ]: fruits = ["Apple", "Banana", "Mango", "Banana"]  
  
print(fruits.index("Banana"))
```

Output

1

Explanation

Although "Banana" appears twice, `index()` returns the position of its **first occurrence**.

2. count()

The `count()` method returns the number of times a value appears in a list.

Syntax

```
list_name.count(value)
```

Example

```
In [ ]: numbers = [10, 20, 10, 30, 10]  
  
print(numbers.count(10))
```

Output

3

Explanation

The value 10 appears three times.

3. sort()

The `sort()` method sorts the original list.

By default, it sorts the list in **ascending order**.

Syntax

```
list_name.sort()
```

Example

```
In [ ]: numbers = [40, 10, 60, 20]

numbers.sort()

print(numbers)
```

Output

```
[10, 20, 40, 60]
```

Descending Order

```
In [ ]: numbers = [40, 10, 60, 20]

numbers.sort(reverse=True)

print(numbers)
```

Output

```
[60, 40, 20, 10]
```

4. reverse()

The `reverse()` method reverses the order of the elements in the list.

Syntax

```
list_name.reverse()
```

Example

```
In [ ]: numbers = [10, 20, 30, 40]

numbers.reverse()

print(numbers)
```

Output

```
[40, 30, 20, 10]
```

Explanation

The order of the list is reversed.

5. copy()

The `copy()` method creates a **shallow copy** of the list.

Syntax

```
new_list = list_name.copy()
```

Example

```
In [ ]: numbers = [10, 20, 30]

        copy_numbers = numbers.copy()

        print(copy_numbers)
```

Output

```
[10, 20, 30]
```

Explanation

A new list is created containing the same elements as the original list.

sort() vs sorted()

sort()	sorted()
List method	Built-in function
Modifies the original list	Returns a new sorted list
Works only with lists	Works with any iterable
Returns <code>None</code>	Returns a sorted list

Example

```
In [ ]: numbers = [40, 10, 60, 20]

        print(sorted(numbers))
        print(numbers)
```

Output

```
[10, 20, 40, 60]
[40, 10, 60, 20]
```

Complete List Methods Reference

Method	Purpose
<code>append(x)</code>	Add one element at the end.

Method	Purpose
<code>insert(i, x)</code>	Insert an element at index <code>i</code> .
<code>extend(iterable)</code>	Add multiple elements.
<code>remove(x)</code>	Remove the first occurrence of <code>x</code> .
<code>pop([i])</code>	Remove and return an element.
<code>clear()</code>	Remove all elements.
<code>index(x)</code>	Return the first index of <code>x</code> .
<code>count(x)</code>	Count occurrences of <code>x</code> .
<code>sort()</code>	Sort the list.
<code>reverse()</code>	Reverse the list.
<code>copy()</code>	Create a shallow copy.

Key Points

- List methods are called using the **dot (.) operator**.
- `index()` returns the first occurrence of a value.
- `count()` counts how many times a value appears.
- `sort()` modifies the original list.
- `reverse()` changes the order of elements.
- `copy()` creates a shallow copy of the list.
- Use `sorted()` if you want a sorted copy while keeping the original list unchanged.

Copying Lists

Sometimes you need to create a copy of a list instead of working with the original list.

Python provides several ways to copy a list.

The most common methods are:

- Assignment (`=`)
- `copy()` method
- Slicing (`[ : ]`)

Note: Assignment (`=`) does **not** create a new list. It creates another reference to the same list.

1. Using Assignment (`=`)

When you assign one list to another using `=` , both variables refer to the **same list**.

Example

```
In [ ]: list1 = [10, 20, 30]

list2 = list1

list2[0] = 100

print("List 1:", list1)
print("List 2:", list2)
```

Output

```
List 1: [100, 20, 30]
List 2: [100, 20, 30]
```

Explanation

- `list1` and `list2` refer to the same list.
 - Changing one list also changes the other.
-

2. Using the `copy()` Method

The `copy()` method creates a **shallow copy** of the list.

The copied list is a different object, so modifying it does not affect the original list.

Syntax

```
new_list = list_name.copy()
```

Example

```
In [ ]: list1 = [10, 20, 30]

list2 = list1.copy()

list2[0] = 100

print("List 1:", list1)
print("List 2:", list2)
```

Output

```
List 1: [10, 20, 30]
List 2: [100, 20, 30]
```

Explanation

`copy()` creates a new list with the same elements.

3. Using List Slicing (`[ : ]`)

A list can also be copied using slicing.

Syntax

```
new_list = list_name[:]
```

Example

```
In [ ]: list1 = [10, 20, 30]

list2 = list1[:]

list2[1] = 200

print("List 1:", list1)
print("List 2:", list2)
```

Output

```
List 1: [10, 20, 30]
List 2: [10, 200, 30]
```

Explanation

`[:]` creates a new list containing the same elements.

Assignment vs `copy()` vs Slicing

Method	Creates a New List	Changes Affect Original List
<code>=</code>	✗ No	✓ Yes
<code>copy()</code>	✓ Yes	✗ No
<code>[:]</code>	✓ Yes	✗ No

Which Method Should You Use?

Method	Recommended Use
<code>=</code>	When both variables should refer to the same list.
<code>copy()</code>	Best way to create a copy of a list.
<code>[:]</code>	Quick way to create a copy using slicing.

Key Points

- `=` creates another reference to the same list.
- `copy()` creates a shallow copy of the list.
- `[:]` also creates a shallow copy.
- Modifying a copied list does not affect the original list.
- Use `copy()` or `[:]` when you need an independent copy of a list.

Joining Lists

Joining lists means combining two or more lists into a single list.

Python provides several ways to join lists.

The most common methods are:

- Using the `+` operator
- Using the `extend()` method
- Using a `for` loop

Note: Choose the method based on whether you want to create a new list or modify an existing list.

1. Using the `+` Operator

The `+` operator combines two or more lists and returns a **new list**.

Syntax

```
new_list = list1 + list2
```

Example

```
In [ ]: list1 = [1, 2, 3]
        list2 = [4, 5, 6]

        result = list1 + list2

        print(result)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation

The `+` operator joins both lists and creates a new list. The original lists remain unchanged.

2. Using the `extend()` Method

The `extend()` method adds all elements of one list to another.

Syntax

```
list1.extend(list2)
```

Example

```
In [ ]: list1 = [1, 2, 3]
        list2 = [4, 5, 6]

        list1.extend(list2)
```

```
print(list1)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation

The elements of `list2` are added to `list1`.

Unlike the `+` operator, `extend()` modifies the original list.

3. Using a `for` Loop

You can also join lists by adding each element individually using a loop.

Example

```
In [ ]: list1 = [1, 2, 3]
        list2 = [4, 5, 6]

        for item in list2:
            list1.append(item)

        print(list1)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation

The loop iterates through `list2` and adds each element to `list1` using `append()`.

Comparison of Joining Methods

Method	Creates a New List	Modifies Original List
<code>+</code>	✓ Yes	✗ No
<code>extend()</code>	✗ No	✓ Yes
<code>for</code> loop + <code>append()</code>	✗ No	✓ Yes

Which Method Should You Use?

Method	Best Use
<code>+</code>	When you want a new combined list.
<code>extend()</code>	When you want to add elements to an existing list.
<code>for</code> loop	When you need additional processing while joining lists.

Operation	Result
<code>list1 + list2</code>	Creates a new list
<code>list1.append(list2)</code>	Adds list2 as a single nested element
<code>list1.extend(list2)</code>	Adds each element of <code>list2</code> individually

Key Points

- The `+` operator joins lists and returns a new list.
- The `extend()` method adds all elements of one list to another.
- A `for` loop with `append()` can also be used to join lists.
- `extend()` and the `for` loop modify the original list.
- The `+` operator keeps the original lists unchanged.

Nested Lists

A **nested list** is a list that contains one or more lists as its elements.

In other words, a list inside another list is called a **nested list**.

Nested lists are useful for storing data in a table-like structure, such as student records, marks, or matrices.

Syntax

```
list_name = [[item1, item2], [item3, item4]]
```

Example 1: Creating a Nested List

```
In [ ]: students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]  
  
print(students)
```

Output

```
[['Alice', 20], ['Bob', 21], ['Charlie', 22]]
```

Explanation

The main list contains three sublists.

Each sublist stores the name and age of a student.

Structure of a Nested List

```
students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]
```

Main Index	Sublist
0	["Alice", 20]
1	["Bob", 21]
2	["Charlie", 22]

Accessing Nested List Elements

To access an element inside a nested list, use **multiple indexes**.

Syntax

```
list_name[outer_index][inner_index]
```

- `outer_index` → Selects the sublist.
- `inner_index` → Selects the element inside the sublist.

Example 2: Accessing a Sublist

```
In [ ]: students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]  
  
print(students[1])
```

Output

```
['Bob', 21]
```

Example 3: Accessing an Individual Element

```
In [ ]: students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]  
  
print(students[1][0])
```

Output

```
Bob
```

Explanation

- `students[1]` selects the second sublist.

- `[0]` selects the first element of that sublist.
-

Example 4: Accessing Another Element

```
In [ ]: students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]  
  
print(students[2][1])
```

Output

22

Modifying Nested List Elements

Nested list elements can be modified using indexes.

Example

```
In [ ]: students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]  
  
students[1][0] = "David"  
  
print(students)
```

Output

[['Alice', 20], ['David', 21], ['Charlie', 22]]

Explanation

The name `"Bob"` is replaced with `"David"`.

Looping Through a Nested List

Nested loops are commonly used to access every element.

Example

```
In [ ]: students = [  
    ["Alice", 20],  
    ["Bob", 21],  
    ["Charlie", 22]  
]  
  
for row in students:  
    for item in row:  
        print(item)
```

Output

Alice
20
Bob
21
Charlie
22

Explanation

- The outer loop accesses each sublist.
- The inner loop accesses each element inside the sublist.

Real-Life Example

```
In [ ]: marks = [  
    ["Alice", 85],  
    ["Bob", 92],  
    ["Charlie", 78]  
]  
  
for student in marks:  
    print(student)
```

Output

['Alice', 85]
['Bob', 92]
['Charlie', 78]

Summary Table

Concept	Example
Create a nested list	[[1, 2], [3, 4]]
Access a sublist	list[0]
Access an element	list[0][1]
Modify an element	list[1][0] = value
Loop through nested lists	Nested for loops

Key Points

- A nested list is a list containing one or more lists.
- Access nested elements using multiple indexes.
- Nested lists are mutable, so their elements can be modified.
- Nested loops are commonly used to iterate through nested lists.
- Nested lists are useful for representing tables, matrices, and structured data.

Introduction to Matrix

A **matrix** is a rectangular arrangement of data organized into **rows** and **columns**.

In Python, a matrix is commonly represented using a **nested list**, where each inner list represents a row of the matrix.

Note: Python does not have a built-in matrix data type. Beginners usually represent matrices using nested lists. For advanced matrix operations, the **NumPy** library is commonly used.

Example of a Matrix

```
In [ ]: matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(matrix)
```

Output

```
[[1, 2, 3],  
 [4, 5, 6],  
 [7, 8, 9]]
```

Matrix Representation

```
1  2  3  
4  5  6  
7  8  9
```

- Number of rows = **3**
- Number of columns = **3**

Accessing Matrix Elements

Since a matrix is a nested list, elements are accessed using two indexes.

Syntax

```
matrix[row][column]
```

Example

```
In [ ]: matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(matrix[1][2])
```

Output

Explanation

- `matrix[1]` selects the second row: `[4, 5, 6]`
 - `matrix[1][2]` selects the third element of that row, which is `6`.
-

Applications of Matrices

Matrices are widely used in:

- Mathematics
 - Computer Graphics
 - Machine Learning
 - Data Science
 - Image Processing
 - Game Development
-

Key Points

- A matrix is a collection of rows and columns.
- In Python, a matrix is represented using a nested list.
- Each inner list represents one row.
- Elements are accessed using `matrix[row][column]`.
- For advanced matrix operations, the **NumPy** library is commonly used.

List Comprehension (Introduction)

A **list comprehension** is a short and concise way to create a new list in Python.

It allows you to generate a list using a single line of code instead of writing multiple lines with a loop.

Note: List comprehensions make code shorter and more readable. However, they should only be used when they improve readability.

Without List Comprehension

```
In [ ]: numbers = []

for i in range(5):
    numbers.append(i)

print(numbers)
```

Output

`[0, 1, 2, 3, 4]`

With List Comprehension


```
In [ ]: numbers = [i for i in range(5)]  
  
print(numbers)
```

Output

```
[0, 1, 2, 3, 4]
```

Explanation

Both programs produce the same result, but the list comprehension version is shorter.

Syntax

```
new_list = [expression for item in iterable]
```

Syntax Explanation

- **expression** → The value to be added to the new list.
 - **item** → Variable used to access each element.
 - **iterable** → A sequence such as a list, tuple, string, or `range()`.
-

Example 1: Create a List of Numbers

```
In [ ]: numbers = [x for x in range(1, 6)]  
  
print(numbers)
```

Output

```
[1, 2, 3, 4, 5]
```

Example 2: Square of Numbers

```
In [ ]: squares = [x * x for x in range(1, 6)]  
  
print(squares)
```

Output

```
[1, 4, 9, 16, 25]
```

Explanation

Each number is squared before being added to the list.

List Comprehension with `if`

A condition can be used to include only specific elements.

Syntax

```
new_list = [expression for item in iterable if condition]
```

Example

```
In [ ]: even_numbers = [x for x in range(1, 11) if x % 2 == 0]

print(even_numbers)
```

Output

```
[2, 4, 6, 8, 10]
```

Explanation

Only even numbers are added to the list.

List Comprehension with `if...else`

You can also use `if...else` inside a list comprehension.

Syntax

```
new_list = [expression1 if condition else expression2 for item in iterable]
```

Example

```
In [ ]: result = ["Even" if x % 2 == 0 else "Odd" for x in range(1, 6)]

print(result)
```

Output

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Explanation

For each number:

- If it is even, `"Even"` is added.
 - Otherwise, `"Odd"` is added.
-

Advantages of List Comprehension

- Shorter and cleaner code.
 - Easy to create new lists.
 - More readable for simple operations.
 - Often faster than using a `for` loop.
-

Limitations

- Can become difficult to read if too complex.
 - Not suitable for large or complicated logic.
 - Use a normal `for` loop when readability is more important.
-

Summary Table

Feature	Example
Basic	<code>[x for x in range(5)]</code>
Expression	<code>[x * x for x in range(5)]</code>
With <code>if</code>	<code>[x for x in range(10) if x % 2 == 0]</code>
With <code>if...else</code>	<code>["Even" if x % 2 == 0 else "Odd" for x in range(5)]</code>

Key Points

- List comprehension creates a new list in a single line.
- It is an alternative to using a `for` loop with `append()`.
- It can include expressions and conditions.
- Use list comprehensions for simple and readable code.
- Use a normal loop for complex logic.

Common Errors

- Using an invalid index (`IndexError`).
 - Confusing `append()` and `extend()`.
 - Using `remove()` for an element that does not exist (`ValueError`).
 - Forgetting that list indexing starts from `0`.
 - Using `=` instead of `copy()` when copying a list.
 - Mixing up `sort()` and `sorted()`.
 - Incorrect slicing (`start:stop:step`).
 - Modifying a list while iterating over it.
-

Best Practices

- Use meaningful list variable names.
- Use `append()` to add one element and `extend()` to add multiple elements.
- Use `copy()` or `[:]` to create an independent copy of a list.
- Check that an index exists before accessing it.
- Use list comprehensions for simple and readable code.
- Keep list operations simple and easy to understand.
- Avoid modifying a list while looping through it.
- Choose the appropriate list method for the task.



Coding Questions (10)

- 1. Write a program to create a list of five integers and print it.
- 2. Write a program to access and print the first and last element of a list.
- 3. Write a program to replace the third element of a list with a new value.
- 4. Write a program to add an element to the end of a list using `append()`.
- 5. Write a program to insert an element at a specific position in a list.
- 6. Write a program to remove an element from a list using `remove()`.
- 7. Write a program to find the largest and smallest element in a list.
- 8. Write a program to count how many times a given element appears in a list.
- 9. Write a program to sort a list in ascending and descending order.
- 10. Write a program to merge two lists and display the combined list.



Practice Programs (10)

- 1. Write a program to calculate the sum of all elements in a list.
- 2. Write a program to find the average of elements in a list.
- 3. Write a program to find the maximum and minimum element in a list without using `max()` and `min()`.
- 4. Write a program to count the number of even and odd numbers in a list.
- 5. Write a program to search for an element in a list and display its index.
- 6. Write a program to remove duplicate elements from a list while preserving the original order.
- 7. Write a program to reverse a list without using the `reverse()` method.
- 8. Write a program to find the second largest element in a list.
- 9. Write a program to create a multiplication table (1 to 10) and store each table as a nested list.
- 10. Create a menu-driven program to perform the following list operations:
 - Create a List
 - Display the List
 - Add an Element
 - Insert an Element
 - Remove an Element
 - Search for an Element
 - Sort the List
 - Reverse the List
 - Exit



Cheat Sheet

Create a List

```
numbers = [10, 20, 30]
fruits = ["Apple", "Banana", "Mango"]
```

Access Elements

```
numbers[0]      # First element
numbers[-1]     # Last element
numbers[1:4]    # Slicing
numbers[::-1]   # Reverse
```

Modify Elements

```
numbers[1] = 100
numbers[1:3] = [50, 60]
```

Add Elements

```
numbers.append(40)
numbers.insert(1, 15)
numbers.extend([50, 60])
```

Remove Elements

```
numbers.remove(20)
numbers.pop()
numbers.pop(1)
del numbers[0]
numbers.clear()
```

Loop Through a List

```
for item in numbers:
    print(item)

for i in range(len(numbers)):
    print(numbers[i])

for index, value in enumerate(numbers):
    print(index, value)
```

List Operations

```
list1 + list2
list1 * 2
10 in numbers
20 not in numbers
```

Built-in Functions

```
len(numbers)
min(numbers)
max(numbers)
sum(numbers)
sorted(numbers)
```

```
any(numbers)
all(numbers)
```

List Methods

```
append()
insert()
extend()
remove()
pop()
clear()
index()
count()
sort()
reverse()
copy()
```

Copy Lists

```
list2 = list1.copy()
list2 = list1[:]
```

Join Lists

```
list3 = list1 + list2

list1.extend(list2)
```

Nested List

```
matrix = [
    [1, 2],
    [3, 4]
]

matrix[1][0]
```

List Comprehension

```
[x for x in range(5)]

[x*x for x in range(5)]

[x for x in range(10) if x % 2 == 0]
```



Summary

- A **list** is an ordered, mutable collection that allows duplicate elements.
- Lists are created using square brackets `[]`.

- Elements are accessed using indexing and slicing.
- Lists can be modified, expanded, and reduced dynamically.
- Use `append()` , `insert()` , and `extend()` to add elements.
- Use `remove()` , `pop()` , `del` , and `clear()` to remove elements.
- Lists can be traversed using `for` loops, `while` loops, `range()` , or `enumerate()` .
- Python provides useful built-in functions such as `len()` , `min()` , `max()` , `sum()` , and `sorted()` .
- List methods simplify searching, sorting, copying, and modifying lists.
- Nested lists can represent tables or matrices.
- List comprehensions provide a concise way to create new lists.

Next Topic:  **Day 8 – Tuples, Sets & Dictionaries**